

Noten-Fritze

Lokale Noten- und Mitarbeiterverwaltung für Lehrkräfte an Gymnasien. **100 % lokal** im Browser (IndexedDB), **keine Cloud, kein Server, kein Login**. Tablet-first, optimiert für das **iPad im Querformat**, offline nutzbar als PWA.

Verfügbarkeit & Hosting

Die App ist online verfügbar unter:

- <http://noten-fritze.patrick-knapp.de> (Domain-Redirect)
- Gehostet auf GitHub Pages: https://pak-x-nrw.github.io/Noten_Fritze/

Wichtig: Alle Daten bleiben auf Ihrem Gerät! Die App speichert nichts in der Cloud. Sie können die App als PWA installieren („Zum Home-Bildschirm hinzufügen“) und nutzen sie offline.

1. Architektur & Begründung

Vanilla JS + IndexedDB, ohne Build-Schritt, ohne Abhängigkeiten.

- **Warum kein Framework?** Die App ist im Kern eine formular- und tabellenlastige Datenverwaltung mit klar abgegrenzten Screens. Das lässt sich mit einer schlanken, zustandsgesteuerten Render-Schleife (~1 Datei) robust abbilden. Kein Framework bedeutet: keine Toolchain, keine Versions-Updates, **läuft in 10 Jahren noch**, öffnet sich per Doppelklick. Genau das passt zur Anforderung „einfache, langlebige Technologie“.
- **Klassische `<script>`-Tags statt ES-Module**, damit die App auch direkt per `file://` (Doppelklick auf `index.html`) läuft – ES-Module würden dort an CORS scheitern. Modularität entsteht über getrennte Dateien und Namespaces (`DB`, `Store`, `Calc`, `CSV`, `UI`, `Views`).
- **Datenhaltung:** IndexedDB für alle Nutzdaten (asynchron, große Mengen, strukturierte Objekte). LocalStorage wird bewusst **nicht** für Nutzdaten benutzt.
- **Schichten:**
 - `db.js` – generischer IndexedDB-Wrapper (Promises, Schema-Versionierung)
 - `store.js` – Domänenmodell, Repositories, Defaults, Demo-Daten
 - `calc.js` – reine Rechenlogik (Noten, Mitarbeit, Farben) – frei von DOM/DB
 - `csv.js` – CSV-Export/Import
 - `ui.js` – UI-Bausteine (Modal, Toast, Formfelder)
 - `views.js` – Screens, Routing, Interaktion
 - `app.js` – Bootstrap
- **PWA:** `manifest.webmanifest` + `service-worker.js` (App-Shell-Cache) für Offline-Betrieb und „Zum Home-Bildschirm hinzufügen“. Der Service Worker aktiviert sich nur beim Hosting über http(s), nicht unter `file://`.

2. Tech-Stack

Bereich	Wahl	Begründung
Sprache	HTML/CSS/Vanilla JS	langfristig, keine Toolchain
Speicher	IndexedDB	lokal, asynchron, strukturiert
UI-Einstellung	(LocalStorage frei)	nur für Kleinkram vorgesehen
Offline	Service Worker + Manifest	PWA-Grundlagen ohne Ballast
Build	keiner	statische Dateien, sofort lauffähig

3. Datenmodell (IndexedDB-Stores)

```

klassen      { id, name, schuljahr, fach, typ('hauptfach'|'nebenfach'),
               anteilSchriftlich, anteilSonstige, notizen,
               createdAt, updatedAt, lastOpenedAt }
schueler     { id, klasseId, vorname, nachname, bemerkung, sortIndex, ... }
kategorien   { id, klasseId, name, art('schriftlich'|'sonstige'),
               gewichtung, sortIndex }
noten        { id, klasseId, schuelerId, kategorieId, wert(1..6),
               titel, datum(YYYY-MM-DD), createdAt }
sitzplaene   { klasseId, rows, cols, seats:[{id,row,col,schuelerId}] }
ereignisse   { id, klasseId, schuelerId, typ, punkte, timestamp, notiz }
einstellungen { key:'app', rundung, mitarbeitPunkte, mitarbeitSchwellen,
               heatPunkteEinfach, heatPunkteGut, heatPunkteSehrGut,
               heatStartWert, heatVerfallPunkte, heatVerfallMinuten, anteile }

```

- **Integrität:** Löschen einer Klasse/eines Schülers löscht kaskadierend alle abhängigen Datensätze (Noten, Ereignisse, Sitzplatz-Zuweisung).
- **Versionierbarkeit:** `DB_VERSION` + `onupgradeneeded` legen Stores/Indizes an; spätere Migrationen additiv ergänzen. Zusätzlich JSON-Voll-Backup als Sicherung.

4. Rechenlogik – gewichtete Gesamtnote

Zwei-Ebenen-Gewichtung (passend zur Hauptfach/Nebenfach-Unterscheidung):

1. **Kategorie-Ø:** Durchschnitt der Einzelnoten je Kategorie (leere ignoriert).
2. **Gruppen-Ø:** je Art-Gruppe („schriftlich“, „sonstige“) gewichteter Mittelwert der Kategorie-Durchschnitte über deren `gewichtung`.
3. **Gesamtnote:** `anteilSchriftlich % · Ø_schriftlich + anteilSonstige % · Ø_sonstige`.

Robustheit: Fehlt eine ganze Gruppe (keine Noten), zählt die vorhandene Gruppe zu 100 %. Kategorien ohne Noten fallen aus der Gewichtung heraus, statt als „0“ zu verfälschen. Fehlt alles, ist die Gesamtnote „–“.

Rundung (Einstellung): eine Nachkommastelle (Standard), zwei Nachkommastellen oder ganze Note.

Noteneingabe akzeptiert `2`, `2,3`, `2.3`, `2+` (→ 1,7), `2-` (→ 2,3). Die Aufschlüsselung ist im **Besprechungsmodus** und im Schüler-Detail transparent sichtbar (Kategorie-Ø, Gewichte, Gruppen-Ø, Gesamt).

5. Mitarbeits-Tracker & Punktesystem

- **Ereignistypen** (Standardpunkte, konfigurierbar in den Einstellungen):

Typ	Punkte
Wortmeldung	+1
Gute Meldung	+2
Sehr gute Meldung	+3
Störung	-2
Fehlende HA	-1

- **Erfassung:** Jede Schüler-Kachel trägt die 5 Ereignis-Buttons direkt in sich (farbcodiert, mit Tages-Zähler je Typ). Ein Tap auf den passenden Button erzeugt sofort das Ereignis mit Zeitstempel – kein Umweg über eine Auswahlleiste. **Undo** über Toast oder Button.
- **Heatmap:** Farbe je Platz fließend grün→rot, aber **unabhängig** von den Mitarbeitspunkten. Dafür läuft ein eigenes Heatmap-Punktesystem pro Schüler/in: Wortmeldung / Gute / Sehr gute Meldung bringen jeweils frei einstellbare Punkte; zusätzlich lässt sich einstellen, wie viele Heatmap-Punkte pro X Minuten verfallen. Zusätzlich gibt es einen einstellbaren Default-Wert für neue bzw. zurückgesetzte Heatmaps. Der Wertebereich liegt immer bei 0 bis 100. Negative Meldungen wie Störung oder fehlende HA beeinflussen die Heatmap nicht.
- **Epochalnote (Vorschlag):** Summe der Punkte / Anzahl aktiver Tage = Ø-Punkte/Tag, gemappt über konfigurierbare Schwellen auf eine Note 1–6. Bewusst als **Vorschlag** markiert (Auswertungs-Tab, pro Zeitraum: Gesamt / 30 Tage / 7 Tage).

6. CSV-Schema

Format: UTF-8 **mit BOM** (Umlaute in Numbers/Excel korrekt), Trennzeichen `,`, robustes Quoting ("`"` verdoppelt). Der Import erkennt `,` und `;` automatisch.

- **Schülerliste** (`schueler_<Klasse>.csv`): Vorname, Nachname, Bemerkung
- **Noten (breit)** (`noten_<Klasse>.csv`): Vorname, Nachname, <Kategorie> (Ø)…, Gesamtnote
- **Einzelnoten (lang)** (`einzelnoten_<Klasse>.csv`): Vorname, Nachname, Kategorie, Art, Titel, Note, Datum
- **Mitarbeit** (`mitarbeit_<Klasse>.csv`): Vorname, Nachname, Ereignistyp, Punkte, Zeitpunkt
- **Import:** Schülerlisten per CSV-Datei **oder** eingefügter Namensliste („Nachname, Vorname“ bzw. „Vorname Nachname“).
- **Voll-Backup:** JSON über alle Stores (Einstellungen inkl.), Import mit optionalem „vorher alles löschen“.

7. UI-Struktur & Screens

- **Home** – Klassenübersicht als Karten, sortiert nach zuletzt geöffnet; lange nicht geöffnete Klassen mit rotem Punkt + Hinweis. „+ Klasse“, Einstellungen.
- **Klasse** – Tabs: *Schüler/innen* · *Noten* · *Kategorien* · *Sitzplan* · *Mitarbeit*. Oben schnell erreichbar: **Tracker**, **Besprechung**, Bearbeiten.
- **Noten** – Matrix Schüler × Kategorie mit farbigen Ø-Badges + Gesamtnote; Zelle antippen → Einzelnoten erfassen; Namen antippen → Berechnung.

- **Sitzplan** – Raster (Reihen/Spalten frei), Plätze antippen zum Zuweisen, „Automatisch belegen“.
- **Tracker** – Kacheln je Schüler/in mit 5 direkten Ereignis-Buttons (Wortmeldung, Gute/Sehr gute Meldung, Störung, Fehlende HA), Tages-Zähler je Typ, Undo, Heatmap-Hintergrund + Legende.
- **Besprechungsmodus** – ein/e Schüler/in einzeln, groß; nur deren Daten sichtbar (Datenschutz bei der Notenbesprechung), Vor/Zurück.
- **Einstellungen** – Rundung, Mitarbeitspunkte, Heatmap-Punktverfall, Backup/Restore, Demo-Daten, alles löschen.

8. MVP-Funktionsumfang

Enthalten (funktionsfähig): Klassen/Schüler/Kategorien CRUD · Einzelnoten & gewichtete Gesamtnote · Sitzplan-Editor · Tracker mit Tap/Undo/Heatmap · Mitarbeits-Auswertung + Notenvorschlag · Besprechungsmodus · CSV-Export (4 Arten) · CSV-Import Schüler · JSON-Voll-Backup · PWA/Offline · Demo-Daten.

Bewusst später (klar als Ausbau markiert): Drag&Drop-Sortierung (aktuell ▲▼-Buttons) · Noten-CSV-Import (nur Export + Schüler-Import) · mehrere Sitzpläne/Perioden je Klasse · frei editierbare Mitarbeitsschwellen im UI (aktuell über Backup-JSON/Defaults) · echte PNG-App-Icons (aktuell SVG) · Mehrbenutzer/Sync (nicht vorgesehen, da lokal).

9. Dateistruktur

```

Noten_Fritze/
├─ index.html           App-Shell
├─ manifest.webmanifest PWA-Manifest
├─ service-worker.js    Offline-Cache
├─ css/styles.css       Styles (iPad-first)
├─ icons/icon.svg       App-Icon
└─ js/
    ├─ db.js            IndexedDB-Wrapper
    ├─ store.js         Datenmodell, Repos, Demo-Daten
    ├─ calc.js          Noten-/Mitarbeitslogik
    ├─ csv.js           CSV-Export/Import
    ├─ ui.js            Modal/Toast/Formfelder
    ├─ views.js         Screens + Routing
    └─ app.js           Bootstrap

```

10. Lokaler Start

Schnell testen (Desktop): `index.html` doppelklicken. Es öffnet sich im Browser, Demo-Daten werden beim ersten Start angelegt. IndexedDB und CSV-Export funktionieren per `file:///`. (Nur die PWA/Offline-Installation braucht http – siehe unten.)

Empfohlen (mit PWA/Offline, für iPad): einen kleinen lokalen Server nutzen und die Adresse im iPad-Chrome/Safari öffnen (gleiches WLAN):

```

# im Projektordner, eine der Varianten:
npx serve .           # Node

```

```
python -m http.server # Python 3 -> http://localhost:8000
```

Auf dem iPad die URL öffnen → Teilen → „**Zum Home-Bildschirm**“. Danach läuft die App im Vollbild und offline. Alle Daten bleiben ausschließlich auf dem Gerät.

Datensicherung: Einstellungen → „Backup exportieren (JSON)“. Da Daten im Browser liegen, gehen sie beim Löschen der Website-Daten verloren – regelmäßig sichern.

11. Datenspeicherung & Backup

Wie funktioniert die Speicherung?

- **Automatisch beim Eingeben:** Jede Änderung (neue Klasse, Schüler, Noten, Mitarbeitseintrag) wird sofort in IndexedDB geschrieben – **keine manuellen Speicherschritte nötig**.
- **Auf dem Gerät selbst:** Die Daten werden lokal im iPad/Browser gespeichert, nicht in der Cloud, nicht auf Strato, nicht auf einem Server. Vollständige Datensouveränität.
- **Dauerhaft:** Die Daten bleiben erhalten nach dem Schließen der App, nach dem Ausschalten des iPads und nach Browser-Neustarts.

Werden Daten beim Schließen gelöscht?

Nein! Die Daten verschwinden **nicht**:

- App schließen → Daten bleiben ✓
- iPad ausschalten → Daten bleiben ✓
- Browser-Tab schließen → Daten bleiben ✓
- Browser deinstallieren → Daten gehen verloren ⚠

Backup-Strategie

Wann sollten Sie exportieren?

Situation	Häufigkeit	Methode
Nach großen Änderungen (neue Klassen, viele Noten)	Monatlich	JSON-Export
Schuljahrs-Ende / Archivierung	1x pro Jahr	JSON-Export + sichern
Tägliche Nutzung (normal)	Nicht nötig	App speichert automatisch
Zusätzliche Sicherheit	Wöchentlich (optional)	JSON-Export

Wie exportieren:

1. **In der App:** Einstellungen → „**Backup exportieren**“ (oder CSV-Export)
2. **Datei speichern:** z.B. `noten-fritze-backup-2026-07-22.json`
3. **Sichern:** Auf Computer, OneDrive, Google Drive oder USB-Stick kopieren

Wie wiederherstellen:

1. **Einstellungen** → „**Backup importieren**“
2. **JSON-Datei auswählen** → alle Daten werden wiederhergestellt
3. Optional: „Vorher alles löschen“ aktivieren (für Neustart)

Sicherheit

- ✓ **Alle Daten bleiben lokal** – kein Cloud-Upload, keine Übertragung
- ✓ **Keine Abhängigkeit** – funktioniert offline, ohne Internetverbindung
- ✓ **Langlebigkeit** – auch in 10 Jahren noch lesbar (Standard-JSON-Format)
- ✓ **Kontrolle:** Sie entscheiden, wann/ob exportiert wird